DATABASES

Normalization Visual Guide

1NF to BCNF --
kill redundancy without killing performance.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

The Problem: Anomalies

One table holding too much truth corrupts itself.

Update anomaly

change a department's city in 50 rows -- miss one, data lies

Insert anomaly

cannot add a department with no employees yet

Delete anomaly

delete the last employee, lose the department

Normalization splits tables so each fact lives in exactly one place.

STEP 2/6

1NF: Atomic Values

Every cell holds ONE value. No lists, no nested tables.

```
BAD  (1NF violation):
  employees(id, name, phones)  ->  phones = '555-0142, 555-7781'

GOOD (1NF): one row per phone
  employee_phones(emp_id, phone)
    (7, '555-0142')   (7, '555-7781')
```

- Test: can you query 'who has phone X' without string-splitting? If no, not 1NF.
- Repeating groups (phone1, phone2, phone3) are the classic smell.

STEP 3/6

2NF: No Partial Dependencies

Every non-key column must depend on the WHOLE key.

```
BAD:  order_lines(order_id, product_id, product_name, qty)
  key = (order_id, product_id), but product_name needs only product_id

GOOD: split it
  order_lines(order_id, product_id, qty)
  products(product_id, product_name)
```

Partial dependency

product_name depends on part of the key -- update it in one row, the others lie

STEP 4/6

3NF & BCNF: No Transitive Dependencies

Non-key columns must depend on the key, the whole key, nothing but the key.

```
BAD:  employees(id, name, dept_id, dept_city)
  dept_city depends on dept_id, which is not the key (transitive)

GOOD (3NF):
  employees(id, name, dept_id)   departments(dept_id, dept_city)
```

BCNF (stricter than 3NF)

every determinant must be a candidate key -- fixes edge cases 3NF misses

Most production schemas stop at 3NF/BCNF. 4NF/5NF exist but rarely pay off in app databases.

STEP 5/6

Denormalize Deliberately

Break the rules on purpose -- with a sync strategy.

Read-heavy counters

store comment_count on posts; update via trigger or job

Hot join paths

embed author_name in articles to skip the join

Reporting snapshots

flattened tables the warehouse owns outright

Rule: denormalize only measured hot paths, and own the invalidation story (triggers, CDC, or jobs).

STEP ✓/✓

Key Takeaways

Normalization -- Remember

- Anomalies (update/insert/delete) are the disease; normalization is the cure.
- 1NF: atomic cells. 2NF: no partial key dependencies. 3NF/BCNF: no transitive dependencies.
- Each normal form removes a class of redundancy -- stop at 3NF/BCNF for most apps.
- Denormalize only for measured hot paths, and always own the sync/invalidation story.
- A fact should live in exactly one place -- every copy is a future inconsistency.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev